TRƯỜNG ĐẠI HỌC SÀI GÒN (SGU) • KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO TOÀN DIỆN PHẦN 2: PHÂN CẤP CON NGƯỜI & ĐA HÌNH CHIẾT KHẤU

ĐỀ TÀI: QUẢN LÝ CỬA HÀNG THỜI TRANG (OOP JAVA) • PHẦN 2

Phân công: Thành viên 2 | GVHD: Thầy Trần Ngọc Minh Hiếu (tnmhieu@sgu.edu.vn)

TÀI LIỆU CHI TIẾT DÀNH CHO THÀNH VIÊN HỌC & BẢO VỆ VẤN ĐÁP (ĐIỂM 10/10)

> **Phân công phụ trách:** Thành viên 2

> **Mục tiêu:** Nắm vững cấu trúc phân cấp đa tầng của mô hình Con người, bản chất đa hình chiết khấu qua Interface `IGiamGiaKhachHang`, phân biệt code chính và code phụ trợ, trả lời lưu loát các câu hỏi vấn đáp của Thầy Hiếu.

1. DANH MỤC CÁC TẬP TIN (FILES) QUẢN LÝ TRONG PHẦN 2

Phần 2 gồm tổng cộng **8 tập tin mã nguồn (.java)**:

1.1. Các tập tin mã nguồn chính (Core Files) - 7 Lớp

1. `Nguoi.java` : Lớp cơ sở (Base Class) đại diện cho thực thể con người trong hệ thống.
2. `KhachHang.java` : Lớp trung gian kế thừa từ `Nguoi`, quản lý thông tin khách mua hàng.
3. `KhachHangVIP.java` : Lớp con kế thừa từ `KhachHang`, hiện thực chiết khấu **20%** cho khách VIP.
4. `KhachHangThanThiet.java` : Lớp con kế thừa từ `KhachHang`, hiện thực chiết khấu **10%** và quản lý điểm tích lũy.
5. `NhanVien.java` : Lớp con kế thừa từ `Nguoi`, hiện thực chiết khấu nội bộ **50%** cho nhân sự cửa hàng.
6. `NhanVienQuanLy.java` : Lớp con kế thừa từ `NhanVien`, mở rộng quản lý phụ cấp trách nhiệm.
7. `NhaCungCap.java` : Lớp độc lập quản lý các đối tác sản xuất, cung ứng may mặc.

1.2. Tập tin phụ trợ & Giao tiếp (Supporting & Interfaces) - 1 Interface

8. `IGiamGiaKhachHang.java` : Interface định nghĩa phương thức giảm giá theo đối tượng (`applyDiscount`).

2. BẢNG PHÂN LOẠI KĨ THUẬT LẬP TRÌNH: CHÍNH VS PHỤ TRỢ

Phân loại	Tên kĩ thuật	Vị trí xuất hiện trong mã nguồn	Vai trò & Ý nghĩa kỹ thuật
Kỹ thuật chính	**Kế thừa đa tầng (Multi-level Inheritance)**	`Nguoi -> KhachHang -> KhachHangVIP`, `Nguoi -> NhanVien -> NhanVienQuanLy`	Xây dựng cây phả hệ logic: Quản lý kế thừa Nhân viên, VIP kế thừa Khách hàng. Tái sử dụng tối đa mã nguồn.
Kỹ thuật chính	**Đa hình qua Interface (Interface Polymorphism)**	`implements IGiamGiaKhachHang`, hàm `applyDiscount(double)`	Đồng nhất hành vi giảm giá cho các đối tượng khác nhau mà không cần viết hàng loạt lệnh if-else phân loại.

Kỹ thuật chính	**Tính Đóng gói (Encapsulation)**	`private String name, numberPhone` , getter/setter	Bảo vệ dữ liệu cá nhân, ngăn chặn việc gán bừa bãi số điện thoại hoặc mã thẻ từ bên ngoài.
Kỹ thuật chính	**Kiểm tra kiểu & Ép kiểu (Downcasting)**	`person instanceof IGiamGiaKhachHang` trong `HoaDon.java`	Kiểm tra động tại runtime để ép kiểu sang interface và gọi hàm chiết khấu tương ứng.
Kỹ thuật phụ trợ	**Làm sạch dữ liệu (Data Sanitization)**	`.replace(",", " ")` , `.replace(",", "")`	Loại bỏ dấu phẩy trong họ tên và SĐT để bảo toàn định dạng file text `users.txt` .
Kỹ thuật phụ trợ	**Kiểm tra biểu thức chính quy (Regex)**	`matches("^0[0-9]{8,10}$")`	Kiểm tra tính hợp lệ của số điện thoại Việt Nam (bắt đầu bằng số 0, dài từ 9 đến 11 số).
Kỹ thuật phụ trợ	**Định dạng bảng hiển thị Console**	`hienThiDanhSach()` , `hienThiThongTin()`	Đóng khung kẻ viền bảng, căn thẳng cột họ tên, SĐT và chức vụ trên màn hình.

3. PHÂN TÍCH CHI TIẾT TỪNG ĐOẠN CODE: CHÍNH VS PHỤ TRỢ

3.1. Tại tập tin lớp cơ sở `Nguoi.java`

```
public class Nguoi {
    private String name;
    private String numberPhone;

    public Nguoi(String name, String numberPhone) {
        this.name = (name != null) ? name.trim().replace(",", " ") : "";
        this.numberPhone = (numberPhone != null) ? numberPhone.trim().replace(",", "") : "";
    }

    public String getName() { return name; }
    public void setName(String name) {
        if (name != null && !name.trim().isEmpty()) {
            this.name = name.trim().replace(",", " ");
        }
    }

    public String getNumberPhone() { return numberPhone; }
    public void setNumberPhone(String numberPhone) {
        if (numberPhone != null) {
            this.numberPhone = numberPhone.trim().replace(",", "");
        }
    }
}
```

- **Kỹ thuật chính:** Đóng gói thuộc tính `name` và `numberPhone` ở phạm vi `private` . Mọi truy cập hay sửa đổi bắt buộc phải thông qua `getName()` và `setName()` .
- **Kỹ thuật phụ trợ:**
- `(name != null) ? ... : ""` : Chống lỗi `NullPointerException` khi khởi tạo.

- `.replace(",", " ")` : Loại bỏ dấu phẩy để dữ liệu khi ghi vào `users.txt` (dạng CSV) không bị lệch cột dữ liệu.

3.2. Tại Interface `IGiamGiaKhachHang.java`

```
public interface IGiamGiaKhachHang {
    double applyDiscount(double totalPrice);
}
```

- **Kỹ thuật chính:** Khai báo một giao tiếp trừu tượng thuần túy. Bất kỳ thực thể con người nào có quyền được giảm giá trong hệ thống đều phải hiện thực phương thức này.
- **Ý nghĩa:** Tách rời hoàn toàn nghiệp vụ chiết khấu ra khỏi cây kế thừa sinh học, cho phép cả Khách hàng và Nhân viên đều có thể áp dụng chung một cơ chế tính tiền.

3.3. Đoạn Code Khách hàng VIP `KhachHangVIP.java`

```
public class KhachHangVIP extends KhachHang implements IGiamGiaKhachHang {
    private String cardCodeVIP;

    public KhachHangVIP(String name, String numberPhone, String cardCodeVIP) {
        super(name, numberPhone, "VIP");
        this.cardCodeVIP = (cardCodeVIP != null) ? cardCodeVIP.trim().replace(",", " ") : "";
    }

    public String getCardCodeVIP() { return cardCodeVIP; }

    @Override
    public double applyDiscount(double totalPrice) {
        return totalPrice - (totalPrice * 0.2);
    }
}
```

- **Kỹ thuật chính (Kế thừa & Constructor Chaining):**
- `super(name, numberPhone, "VIP")` : Gọi constructor 3 tham số của lớp cha `KhachHang` , vừa truyền họ tên và SĐT, vừa tự động gán loại khách hàng là `"VIP"` .
- `private String cardCodeVIP` : Thuộc tính mở rộng đặc trưng cho khách VIP (mã thẻ VIP).
- **Kỹ thuật chính (Ghi đè phương thức @Override):**
- `applyDiscount` : Thực hiện công thức `totalPrice - (totalPrice * 0.2)` -> Giảm trực tiếp **20%** trên tổng tiền đơn hàng.

3.4. Đoạn Code Khách hàng Thân thiết `KhachHangThanThiet.java`

```
public class KhachHangThanThiet extends KhachHang implements IGiamGiaKhachHang {
    private int diemTichLuy;

    public KhachHangThanThiet(String name, String numberPhone, int diemTichLuy) {
        super(name, numberPhone, "ThanThiet");
        this.diemTichLuy = (diemTichLuy >= 0) ? diemTichLuy : 0;
    }

    public int getDiemTichLuy() { return diemTichLuy; }

    @Override
    public double applyDiscount(double totalPrice) {
        return totalPrice - (totalPrice * 0.10);
```

```
    }
}
```

- **Kỹ thuật chính:**
- `diemTichLuy` : Quản lý điểm thưởng tích lũy qua các lần mua sắm. Ràng buộc `diemTichLuy >= 0` .
- `@Override applyDiscount` : Chiết khấu ưu đãi **10%** cho khách hàng thân thiết.

3.5. Đoạn Code Phân cấp Nhân viên `NhanVien.java` & `NhanVienQuanLy.java`

```
public class NhanVien extends Nguoi implements IGiamGiaKhachHang {
    private String staffID;

    public NhanVien(String name, String numberPhone, String staffID) {
        super(name, numberPhone);
        this.staffID = (staffID != null) ? staffID.trim().replace(",", " ") : "";
    }

    @Override
    public double applyDiscount(double totalPrice) {
        return totalPrice - (totalPrice * 0.5);
    }
}
```

```
public class NhanVienQuanLy extends NhanVien {
    private double phuCapTrachNhiem;

    public NhanVienQuanLy(String name, String numberPhone, String staffID, double phuCapTrachNhiem) {
        super(name, numberPhone, staffID);
        this.phuCapTrachNhiem = (phuCapTrachNhiem >= 0) ? phuCapTrachNhiem : 0;
    }

    public double getPhuCapTrachNhiem() { return phuCapTrachNhiem; }
}
```

- **Kỹ thuật chính (Kế thừa đa tầng & Tái sử dụng phương thức):**
- `NhanVien` kế thừa từ `Nguoi` và cài đặt `IGiamGiaKhachHang` với mức giảm nội bộ là **50%**.
- `NhanVienQuanLy extends NhanVien` : Quản lý kế thừa toàn bộ thuộc tính `staffID` và phương thức giảm giá 50% của `NhanVien` mà không cần viết lại dù chỉ 1 dòng code.
- Thuộc tính riêng `phuCapTrachNhiem` : Bổ sung khoản phụ cấp quản lý cho vị trí quản lý cửa hàng.

3.6. Lớp đối tác `NhaCungCap.java`

- Quản lý các đơn vị may mặc, xưởng dệt may đối tác (`maNCC` , `tenNCC` , `diaChi` , `soDienThoai`).
- Đóng gói phương thức `hienThiThongTin()` in dạng bảng cho Menu 9 trong hệ thống.

4. BỘ CÂU HỎI VẤN ĐÁP CỦA THẦY HIẾU (KÈM DÒNG CODE VÀ CÂU TRẢ LỜI ĐIỂM 10)

> **CÂU HỎI VẤN ĐÁP: Thầy chỉ vào lớp `HoaDon.java` và hỏi: "Đoạn code nào thể hiện việc áp dụng đa hình chiết khấu cho khách hàng và nhân viên? Hãy giải thích cơ chế hoạt động?"**
>
> **TRẢ LỜI MẪU (10 ĐIỂM):** "Dạ thưa Thầy, đó là đoạn code trong phương thức `calculateTotalPrice()` của lớp `HoaDon`:

```java
if (person instanceof IGiamGiaKhachHang) {
total = ((IGiamGiaKhachHang) person).applyDiscount(total);
}
```

Cơ chế hoạt động như sau:

1. Lớp `HoaDon` chỉ giữ một con trỏ kiểu cha là `Nguoi person`.
2. Tại thời điểm chạy (Runtime), máy ảo Java kiểm tra xem đối tượng mà `person` đang trỏ tới có hiện thực interface `IGiamGiaKhachHang` hay không bằng toán tử `instanceof`.
3. Nếu có, Java sẽ kích hoạt cơ chế liên kết động (Dynamic Binding) để gọi hàm `applyDiscount` của đúng đối tượng thực tế đó: Nếu là `KhachHangVIP` sẽ giảm 20%, nếu là `KhachHangThanThiet` giảm 10%, nếu là `NhanVien` hoặc `NhanVienQuanLy` giảm 50%.
4. Nhờ đa hình, chúng em không cần viết hàng loạt lệnh if-else phức tạp để kiểm tra từng loại lớp, giúp hệ thống dễ dàng mở rộng khi có thêm loại khách hàng mới."

CÂU HỎI VẤN ĐÁP: Thầy hỏi: "Tại sao `NhanVienQuanLy` lại kế thừa từ `NhanVien` mà không kế thừa trực tiếp từ `Nguoi`?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy, theo đúng mô hình kế thừa hướng đối tượng:

1. Quản lý bản chất vẫn là một Nhân viên (Quan hệ **IS-A**). Quản lý cũng có mã nhân viên (`staffID`), cũng làm việc trong cửa hàng và cũng được hưởng chế độ giảm giá nội bộ 50% khi mua đồ thời trang.
2. Việc cho `NhanVienQuanLy extends NhanVien` giúp tái sử dụng lại 100% mã nguồn và các hành vi đã có của `NhanVien`, chỉ cần bổ sung thêm thuộc tính đặc thù của chức vụ là `phuCapTrachNhiem`. Nếu kế thừa trực tiếp từ `Nguoi`, chúng em sẽ bị trùng lặp mã nguồn (Code Duplication)."

CÂU HỎI VẤN ĐÁP: Thầy chỉ vào đoạn code nhập SĐT trong `Main.java`: `cPhone.matches("^0[0-9]{8,10}$")` và hỏi: "Biểu thức Regex này có ý nghĩa gì?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy, đây là kỹ thuật kiểm tra tính hợp lệ của số điện thoại:

- `^0`: Bắt buộc chuỗi phải bắt đầu bằng chữ số 0 (quy chuẩn số điện thoại tại Việt Nam).
- `[0-9]{8,10}`: Theo sau số 0 phải là từ 8 đến 10 chữ số (tổng độ dài số điện thoại sẽ từ 9 đến 11 số).
- `$`: Đánh dấu kết thúc chuỗi, không cho phép chứa thêm bất kỳ ký tự chữ cái hay ký tự đặc biệt nào khác.
- Biểu thức này đảm bảo dữ liệu khách hàng lưu trong hệ thống luôn chuẩn xác."

CÂU HỎI VẤN ĐÁP: Thầy hỏi: "Làm thế nào để hệ thống ngăn chặn việc tạo 2 người dùng có cùng một số điện thoại?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy, trong lớp mảng `DanhSachKhachHang`, trước khi thêm một người dùng mới bằng hàm `them(Nguoi person)`, chương trình luôn gọi hàm `timKiemTheoSDT(person.getNumberPhone())`. Nếu kết quả trả về khác `null` (nghĩa là số điện thoại này đã tồn tại trong danh sách), hệ thống sẽ lập tức từ chối thêm và in thông báo lỗi, bảo toàn tính duy nhất của khóa chính định danh người dùng."